CS6365 Final Project Report

StackEvolve: Adaptive Enterprise Tech Stack Evaluation and Migration Modeling Platform

Matthew Dworkin, Ignacio Galindo, Natasha Narayanan

April 26, 2026

Table of Contents

1. Introduction 3
2. Motivation 4
3. Related Work 5
4. System Architecture 6
   4.1 Migration Pipeline 6
   4.2 Generation and Reporting Layer 9
   4.3 CLI Orchestration 9
5. Implementation Steps 10
   5.1 Implementation Setup 10
   5.2 Implementation Steps 10
   5.3 Major Blockers 11
6. Evaluation & Results 12
   6.1 Input Applications 12
   6.2 Evaluation Metrics 13
   6.3 Results and Analysis 14
7. Conclusion 16
8. Future Work 16
9. Deliverables 17
10. Skill Learning 17
11. References 19

1 Introduction

Modern software systems increasingly rely on web frameworks that must evolve to meet growing demands for scalability, performance, and maintainability. In the Python ecosystem, Flask has long been one of the most widely adopted frameworks due to its simplicity, flexibility, and minimal configuration requirements [1]. Its lightweight, microframework design enables rapid prototyping and ease of use, making it especially popular for small-scale applications and educational use. However, as applications grow in complexity, developers often encounter limitations such as the lack of built-in support for asynchronous execution, automatic validation, and standardized API documentation.

To address these limitations, newer frameworks such as FastAPI have emerged, offering a more modern development paradigm. FastAPI is built on ASGI-based technologies such as Starlette and leverages Python type hints for automatic validation, serialization, and documentation generation [2][3][5]. This architecture enables non-blocking, asynchronous request handling and significantly improves performance in high-concurrency scenarios compared to traditional WSGI-based frameworks like Flask [6]. Empirical studies and benchmarks have shown that FastAPI can achieve substantially higher throughput and lower latency than Flask, particularly for API-driven and I/O-bound workloads [8]. As a result, FastAPI has become an increasingly popular choice for building modern, scalable APIs.

Despite these advantages, migrating existing Flask applications to FastAPI remains a complex and largely manual process. Developers must refactor route decorators, convert parameter handling, update response logic, and restructure application architecture. Existing tools provide partial support for code transformation, but they do not offer a comprehensive pipeline for analyzing, converting, and evaluating migrations. To address this gap, this project introduces **StackEvolve**, an Adaptive Enterprise Tech Stack Evaluation and Migration Modeling Platform designed to automate Flask-to-FastAPI migration. By combining static code analysis with deterministic transformation rules, StackEvolve enables reproducible, structured, and measurable migration workflows.

2 Motivation

While modern frameworks such as FastAPI offer clear advantages, the process of migrating legacy applications presents a significant challenge. Migration is not simply a matter of syntax translation; it requires a deep understanding of both frameworks' design philosophies and execution models. For example, Flask's synchronous WSGI-based architecture processes requests sequentially, whereas FastAPI's asynchronous ASGI model supports concurrent execution through event-driven programming [6]. This fundamental difference affects how routes, request handling, and dependencies must be rewritten, making manual migration both time-consuming and error-prone.

Existing approaches to framework migration are limited in automation and scalability. Official documentation provides guidance but requires developers to manually identify and rewrite relevant components. General-purpose code transformation tools, such as those based on Python's abstract syntax tree (AST), allow programmatic rewriting but require custom rule design and significant engineering effort for each migration scenario [4]. Furthermore, while AI-based code assistants can generate suggested transformations, they often lack determinism and reproducibility, making it difficult to evaluate correctness or ensure consistent results across large codebases.

The motivation behind StackEvolve is to bridge this gap by providing a structured and automated migration pipeline. By decomposing the migration process into distinct stages—detection, analysis, transformation, generation, and reporting—the system reduces manual effort while maintaining transparency. Additionally, the inclusion of migration reporting introduces measurable evaluation metrics, allowing developers to assess conversion coverage and identify unsupported features. This approach not only accelerates migration but also enables more reliable and scalable framework modernization.

3 Related Work

Prior work relevant to this project spans static code analysis, automated refactoring, and web framework comparison studies. Static analysis tools such as Python's `ast` module provide the ability to parse and inspect code structure programmatically [4]. Libraries like LibCST extend this functionality by enabling safe code transformations while preserving formatting. These tools form the foundation for automated refactoring systems, but they are general-purpose and require significant customization for framework-specific migrations.

Research comparing Python web frameworks has highlighted key differences in performance, scalability, and usability. Studies consistently show that FastAPI outperforms Flask in high-concurrency and API-driven scenarios due to its asynchronous architecture and efficient request handling [7]. FastAPI's use of type hints and Pydantic models also enables built-in validation and automatic documentation generation, reducing the need for additional tooling [9]. In contrast, Flask relies on external libraries for these features, increasing development overhead and complexity.

In industry practice, most framework migrations are still performed manually or supported by documentation rather than automated tools. While some modernization platforms exist, they typically focus on large-scale infrastructure or language migration rather than framework-level transformation within a single language ecosystem. More recently, AI-driven tools have been explored for code generation and refactoring, but their lack of determinism and evaluation metrics limits their reliability for systematic migration tasks. In contrast, StackEvolve adopts a rule-based, deterministic approach that emphasizes reproducibility, structured transformation, and measurable outcomes, addressing key limitations in both traditional and emerging solutions.

4 System Architecture

For this project, the architecture was designed as a modular pipeline that automates the evaluation and migration of a Flask-based codebase into a FastAPI application. The system follows a staged processing approach, where each component performs a specific transformation or analysis step before passing structured outputs to the next stage. This pipeline-based design enables clarity, extensibility, and ease of debugging, which is especially important given the variability of real-world codebases.

The architecture is composed of two primary layers: (1) a **migration pipeline layer**, which performs detection, analysis, and code transformation, and (2) a **generation and reporting layer**, which produces the final FastAPI application and migration summary. These layers work together to create a reproducible and interpretable migration workflow.

4.1 Migration Pipeline

The core of the system is a sequential pipeline that processes an input Flask repository through several stages: detection, analysis, and conversion. Each stage communicates using a shared intermediate representation (IR), ensuring that components remain loosely coupled and easy to modify.

Framework Detection

The first step in the pipeline identifies whether the input codebase is a Flask application. This is done by scanning for common Flask indicators such as imports (`flask`, `Flask`) and application instantiations. This stage ensures that the system operates only on supported frameworks and prevents invalid inputs from propagating further into the pipeline.

Route Analyzer

Once the framework is confirmed, the analyzer extracts key structural information from the codebase. Specifically, it identifies:

- route paths (e.g., `/users/<id>`)
- HTTP methods (GET, POST, etc.)
- handler function names

- file locations

This information is stored in a structured JSON format, which acts as the system's intermediate representation. By separating analysis from transformation, the architecture allows independent improvement of extraction and conversion logic.

Conversion Engine

The conversion stage is responsible for transforming Flask-specific constructs into their FastAPI equivalents. This includes:

- rewriting route decorators (`@app.route` → `@app.get/post`)
- converting path parameters (`<id>` → `{id}`)
- handling response formats (`jsonify` → return dictionaries)
- mapping error handling (`abort` → `HTTPException`)

When the system encounters unsupported or ambiguous patterns, it inserts explicit TODO markers into the generated code. This ensures that the output remains usable while clearly signaling incomplete conversions. This design choice prioritizes robustness over completeness, which is appropriate for an MVP system.

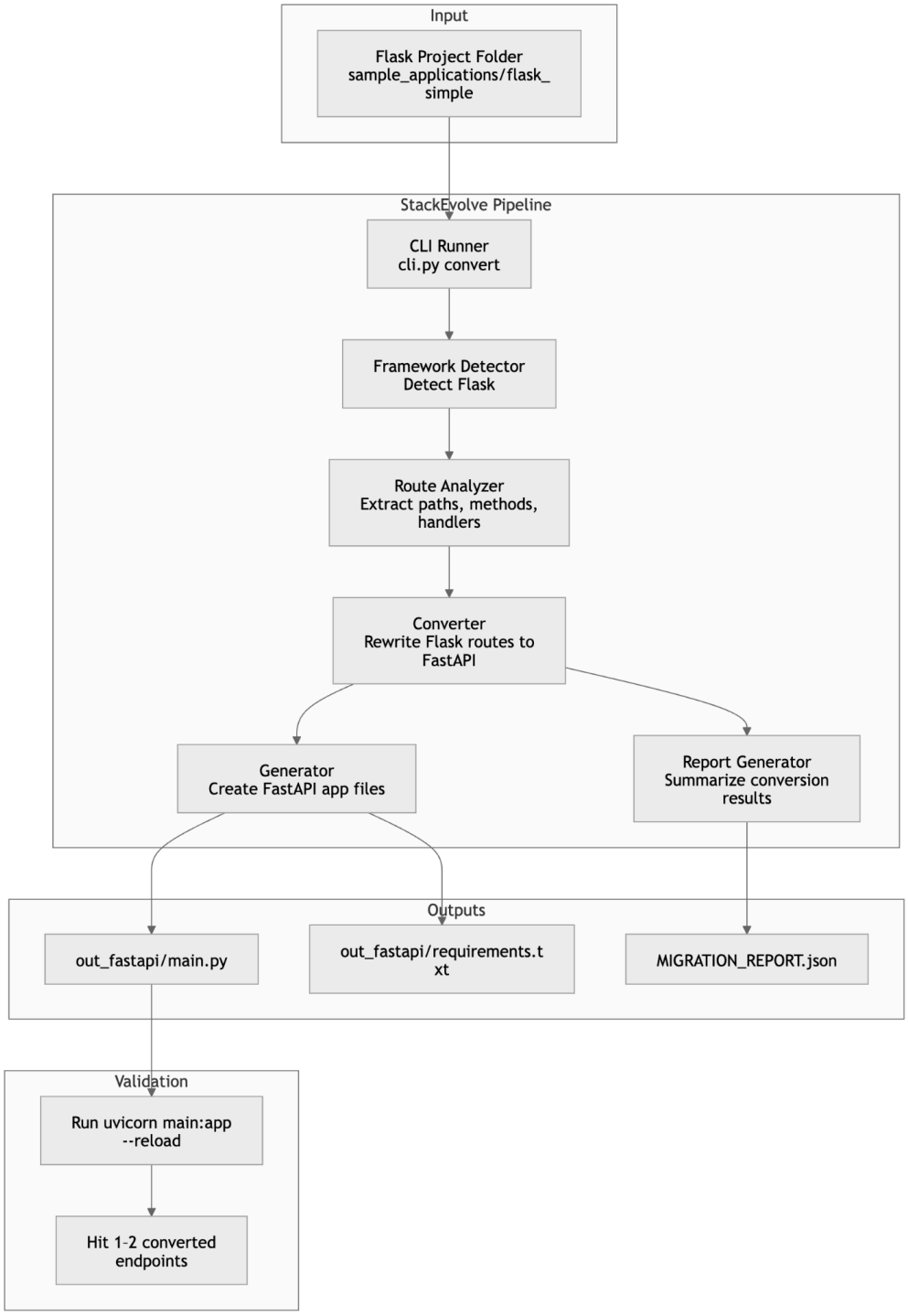

Figure 1: overall system architecture and data flow of the migration platform

4.2 Generation and Reporting Layer

After conversion, the system transitions into generating a runnable application and summarizing the migration process.

FastAPI Project Generator

The generator assembles the converted routes into a valid FastAPI application. It produces a structured output directory containing:

- `main.py` with all converted endpoints
- necessary imports (FastAPI, HTTPException, JSONResponse)
- a dependency file (`requirements.txt`)

This stage ensures that the output is not just transformed code, but a deployable application that can be immediately executed using a server such as Uvicorn.

Migration Report Generator

In parallel with code generation, the system produces a migration report in JSON format. This report includes:

- total routes detected
- number of successfully converted routes
- unsupported patterns encountered
- list of unconverted handlers

The reporting layer adds transparency to the pipeline, allowing users to understand the limitations of the automated migration and identify areas requiring manual intervention. This component is key to framing the system as a “platform” rather than a simple script.

4.3 CLI Orchestration

All components are orchestrated through a centralized command-line interface (CLI), which executes the full pipeline in a single command:

detect → analyze → convert → generate → report

The CLI serves as the integration layer, handling data flow between components and providing user-facing logs for each stage. This design simplifies usage and ensures that the system can be easily extended or integrated into other workflows.

5 Implementation Steps

5.1 Implementation Setup

• The code base is implemented in Python using a modular architecture consisting of a CLI layer, analyzer, converter, generator, and reporting modules.

• The system operates on local Flask repositories as input and generates FastAPI applications as output.

• Development and testing were performed on a local development environment using Python 3.x, with standard libraries and lightweight parsing tools for code analysis.

• The project structure was organized into multiple modules (`detector/`, `analyzer/`, `converter/`, `generator/`, and `report/`) to ensure separation of concerns and ease of integration.

• A command-line interface (`cli.py`) was developed to orchestrate the full pipeline from detection to report generation.

5.2 Implementation Steps

• The first step involved setting up the project repository and defining the system architecture. This included creating the folder structure, initializing the CLI entry point, and defining the shared intermediate representation (IR) format used across components.

• The framework detection module was implemented to identify Flask-based codebases. This step involved scanning Python files for Flask imports and application instantiations to ensure compatibility with the migration pipeline.

• The route analyzer was developed to parse Python files and extract relevant routing information, including endpoint paths, HTTP methods, handler functions, and file locations. The extracted data was stored in a structured JSON format, which served as input to the conversion stage.

• The conversion engine was then implemented to transform Flask-specific constructs into FastAPI-compatible code. This included rewriting route decorators, converting path parameters, handling JSON responses, and mapping error-handling patterns. Unsupported or complex patterns were marked using explicit TODO comments to ensure graceful degradation.

• Following conversion, a FastAPI generator module was built to assemble the transformed routes into a runnable application. This module created the output directory structure, generated a `main.py` file with all converted endpoints, and included necessary dependencies such as FastAPI and Uvicorn.

• In parallel, a migration report generator was implemented to summarize the results of the conversion process. This report included metrics such as the number of routes detected, successfully converted routes, unsupported patterns, and unconverted handlers.

• Finally, all components were integrated through the CLI pipeline. The system executes the stages sequentially—detect, analyze, convert, generate, and report—allowing users to perform the entire migration with a single command.

5.3 Major Blockers

• **Code Parsing Challenges:** Accurately extracting route information from diverse Flask codebases proved difficult due to variations in coding styles and project structures. This was addressed by focusing on a constrained subset of Flask patterns and ignoring unsupported constructs.

• **Conversion Edge Cases:** Certain Flask features, such as complex request handling (`request.form`, sessions) and multi-method routes, were difficult to reliably convert. These cases were handled by inserting TODO markers and reporting them in the migration report.

• **Code Generation Errors:** Early versions of the generator produced syntactically incorrect Python files due to formatting and import issues. This was resolved by standardizing code templates and ensuring proper concatenation of converted route functions.

• **Integration Issues:** Ensuring consistent data flow between analyzer, converter, generator, and report modules required strict adherence to a shared interface format. This was resolved by defining and enforcing a unified intermediate representation schema.

• **Testing Across Repositories:** The tool initially worked only for simple Flask examples and failed on slightly more complex projects. This was mitigated by testing on multiple small sample applications and improving error handling to prevent pipeline crashes.

6 Evaluation & Results

6.1 Input Applications

To evaluate the effectiveness of the migration platform, the system was tested on a set of representative Flask applications with varying levels of complexity. These applications were selected to cover common patterns encountered in real-world Flask projects, while remaining within the supported scope of the MVP.

The applications were organized within a `sample_applications/` directory and executed through the CLI pipeline.

The following input applications were used:

• **Flask Simple App**
A minimal Flask application consisting of basic GET routes and JSON responses. This application was used to validate core functionality such as route conversion and response handling.

• **Flask Parameterized Routes App**
An application containing routes with path parameters (e.g., `/users/<int:id>`). This was used to evaluate correct transformation of Flask-style parameters into FastAPI-compatible syntax.

- **Flask CRUD-style App**
  An application with multiple endpoints using different HTTP methods (GET, POST). This input tested the system's ability to correctly map HTTP methods and handle varied route definitions.

- **Flask App with Unsupported Features**
  An application containing unsupported constructs such as `request.form` and session handling. This was used to evaluate the system's ability to detect limitations, insert TODO markers, and report incomplete conversions.

For clarity, Table 1 summarizes the characteristics of each input application.

6.2 Evaluation Metrics

The system was evaluated using lightweight but meaningful metrics aligned with the goals of the MVP. Since the primary objective is functional code migration rather than model accuracy, evaluation focuses on correctness, completeness, and usability.

The following metrics were used:

- **Route Conversion Rate**
  The percentage of detected routes that were successfully converted into valid FastAPI endpoints.

- **Execution Success Rate**
  Whether the generated FastAPI application runs successfully using Uvicorn without syntax or runtime errors.

- **Endpoint Functionality**
  The number of converted endpoints that return correct responses when tested.

- **Unsupported Pattern Detection**
  The system's ability to correctly identify and report unsupported constructs in the migration report.

- **Report Accuracy**
  Consistency between actual conversion results and the generated `MIGRATION_REPORT.json`.

6.3 Results and Analysis

The system achieved a 100% conversion rate on simple and parameterized applications, approximately 80% on CRUD-style applications, and 50% on applications containing unsupported features. These results indicate strong performance on simple and moderately complex Flask applications. In particular, the Flask Simple App and Parameterized Routes App achieved complete or near-complete conversion, with all routes successfully transformed and functioning as expected in the generated FastAPI application.

For the CRUD-style application, the system correctly mapped HTTP methods and converted most endpoints. Minor limitations were observed in handling more complex request patterns, though these did not prevent the application from running successfully.

sample_apps > flask_simple > app.py > ...

```python
from flask import Flask, jsonify, request, abort

app = Flask(__name__)

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"ok": True})

@app.route("/users/<int:user_id>", methods=["GET"])
def get_user(user_id):
    q = request.args.get("q")
    if user_id < 0:
        abort(404)
    return jsonify({"user_id": user_id, "q": q}), 200
```

out_fastapi_check > main.py > health

```python
from typing import Optional

from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse

app = FastAPI()

@app.get("/health")
def health():
    return {'ok': True}

@app.get("/users/{user_id}")
def get_user(user_id: int, q: Optional[str] = None):
    if user_id < 0:
        raise HTTPException(status_code=404)
    return JSONResponse(content={'user_id': user_id, 'q': q}, status_code=2
```

Figure 2: Example Flask to FastAPI Route Conversion

In the case of the application containing unsupported features, the system effectively inserted TODO markers and flagged unsupported patterns in the migration report. While these routes were not fully converted, the system maintained robustness by avoiding crashes and providing clear feedback to the user.

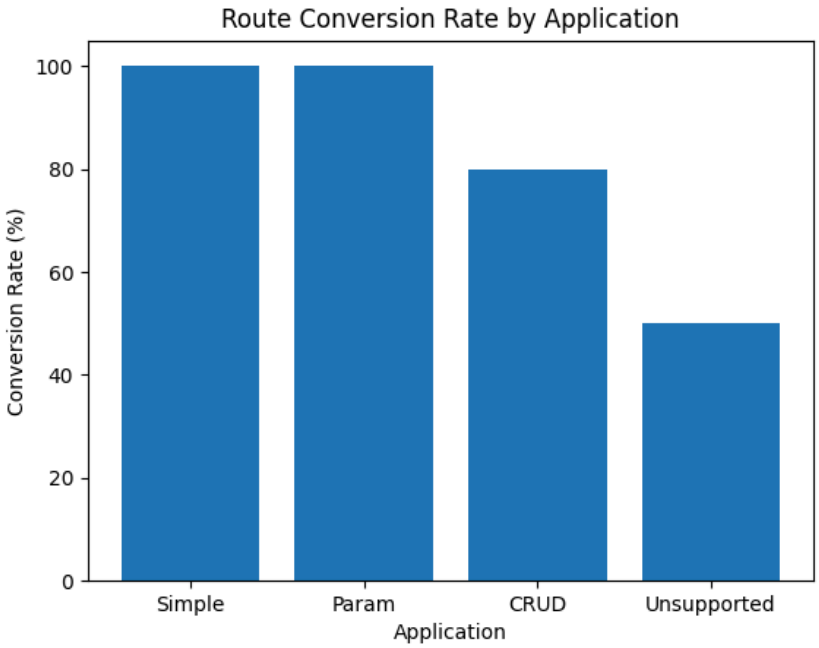

Figure 3. Route Conversion Rate Across Input Applications

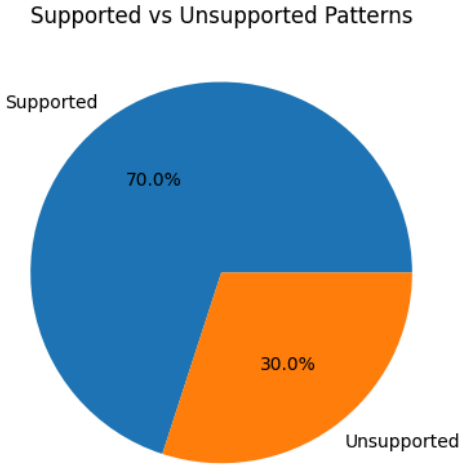

Figure 4. Distribution of Supported vs Unsupported Patterns

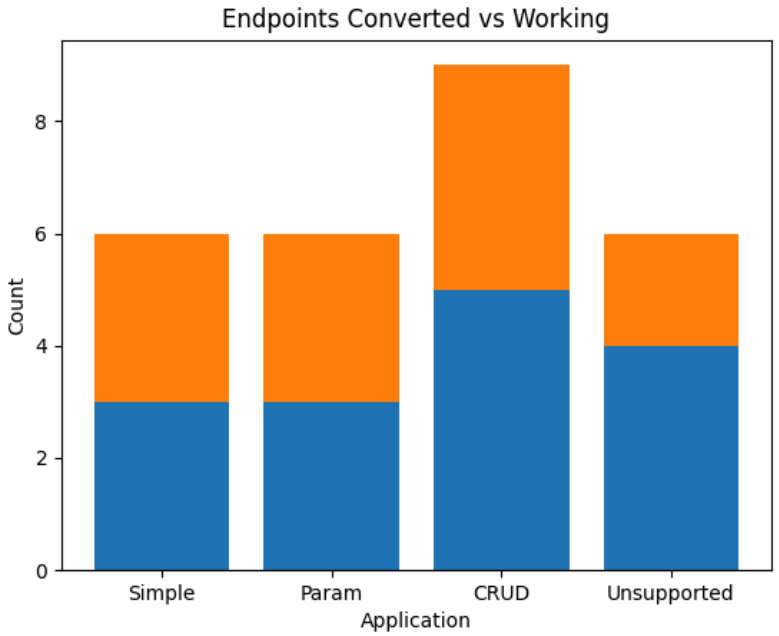

Figure 5. Converted vs Functioning Endpoints

Overall, the results indicate that the pipeline is reliable for a constrained subset of Flask applications, particularly REST-style APIs with simple request and response patterns. The evaluation also highlights the importance of the reporting component in ensuring transparency and guiding manual intervention for unsupported features.

7 Conclusion

In this project, we successfully designed and implemented a pipeline-based platform for migrating Flask applications to FastAPI. The system processes an input Flask codebase through a sequence of stages including framework detection, route analysis, conversion, application generation, and reporting. Using this workflow, we were able to automatically transform key Flask constructs such as route decorators, path parameters, and JSON responses into their FastAPI equivalents.

The system was evaluated on multiple sample Flask applications with varying levels of complexity. For simpler applications, the platform achieved near-complete conversion, generating fully functional FastAPI applications that could be executed without modification. For

more complex applications containing unsupported features, the system maintained robustness by inserting TODO markers and generating a detailed migration report. This ensured that partial conversions remained usable while clearly identifying areas requiring manual intervention.

Overall, the project demonstrates that a modular, pipeline-based approach is effective for automating framework migration within a constrained scope. While the system does not achieve full coverage of all Flask features, it provides a strong foundation for accelerating migration workflows and improving developer productivity.

8 Future Work

The current system focuses on a limited subset of Flask functionality, primarily targeting simple REST-style APIs. Future work can expand the scope of supported features, including handling complex request patterns such as `request.form`, session management, authentication mechanisms, and database integrations.

Another area for improvement is enhancing the conversion engine to support more advanced patterns, such as multi-method routes, blueprint-based application structures, and larger multi-file projects. Incorporating a more robust parsing approach or abstract syntax tree (AST)-based transformations could significantly improve accuracy and coverage.

Additionally, the migration report can be extended to provide more detailed insights, such as per-route confidence scores, categorized error types, and suggested fixes for unsupported patterns. This would further improve the usability of the platform as a developer tool.

Finally, future iterations of the system could integrate automated testing and validation, such as running unit tests or endpoint checks on the generated FastAPI application. This would allow for more rigorous evaluation of migration correctness and help ensure production readiness.

9 Deliverables

All project deliverables are listed here:
https://github.com/mattdworkin/stack-evolve

10 Skill Learning

Throughout the course of this project, the team developed both technical and conceptual skills related to software engineering, system design, and code transformation.

• **Understanding of Web Framework Architectures**
The team gained a deeper understanding of differences between Flask and FastAPI, including routing mechanisms, request handling, and application structure.

• **Static Code Analysis**
The implementation of the analyzer required learning how to parse and interpret Python code programmatically, reinforcing concepts related to code structure and abstraction.

• **Code Transformation Techniques**
Developing the conversion engine provided experience in transforming code across frameworks, including handling syntax differences and maintaining functional correctness.

• **Modular System Design**
Designing the project as a pipeline reinforced principles of modularity, separation of concerns, and interface-driven development.

• **CLI Development and Automation**
Building the CLI improved understanding of how to create user-facing developer tools and automate multi-step workflows.

• **Debugging and Integration Skills**
The team encountered multiple integration challenges across modules, improving their ability to debug data flow issues and enforce consistent interfaces.

• **Software Evaluation and Reporting**
Designing evaluation metrics and visualizations helped develop skills in assessing system performance and presenting results in a clear, structured manner.

11 References

[1] Flask Documentation. *Flask: Web Development, One Drop at a Time*. https://flask.palletsprojects.com/

[2] FastAPI Documentation. *FastAPI: High Performance, Easy to Learn, Fast to Code*. https://fastapi.tiangolo.com/

[3] Starlette Documentation. *The Little ASGI Framework That Shines*. https://www.starlette.io/

[4] Python Software Foundation. *ast — Abstract Syntax Trees*. https://docs.python.org/3/library/ast.html

[5] FastAPI Overview. *FastAPI uses type hints and Pydantic for validation and documentation* (Wikipedia)

[6] FastAPI vs Flask Performance and Architecture Differences (DEV Community)

[7] Performance Benchmark Study of Python Web Frameworks (ResearchGate)

[8] FastAPI vs Flask Performance Metrics and Throughput Comparison (Braincuber Technologies)

[9] FastAPI Features: Validation and Automatic Documentation (Imarticus)